

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Case Study)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO4-AT2-CASE STUDY
Question	Q1 — Personal Finance Management Package in Python (50 Marks)
Section	Section A — Problem Analysis (10 Marks)

Q1. Section A — Problem Analysis [10 Marks]: Analyse the problem of building a Personal Finance Management package in Python.

Problem Statement

Most individuals track income and spending informally — in notebooks, scattered spreadsheets, or bank apps that only cover one account — making it hard to see a unified picture of where money comes from, where it goes, and whether spending is staying within plan. A Personal Finance Management (PFM) package should let a user record income and expenses across multiple accounts, categorise every transaction, compare actual spending against a monthly budget per category, and generate reports that reveal savings rate and top spending areas — all while rejecting bad input (negative amounts, unknown categories, malformed dates) before it corrupts the user's financial picture.

Stakeholders

- **The end user (account holder):** wants a simple, reliable way to log transactions and see where their money is going, without needing to understand the internals.
- **A household/family unit:** may share multiple accounts (e.g. joint bank account, personal wallet) and want combined as well as per-account visibility.
- **A future developer/maintainer:** needs the codebase to be easy to extend — for example adding a new expense category, a new account type, or a new report — without having to rewrite existing, working logic.

Functional Requirements

- **FR1 — Multi-account tracking:** the system must support more than one account (e.g. Bank, Cash) each with its own running balance, updated automatically as transactions are added.
- **FR2 — Transaction logging:** the system must record every transaction with a date, type (Income or Expense), category, amount, description, and the account it affects, and must persist these to a CSV file so they survive between runs.
- **FR3 — Category-based budgeting:** the system must let a monthly spending limit be configured per expense category (via a JSON config file) and must be able to compare actual spend in a given month against that limit.
- **FR4 — Reporting and analytics:** the system must generate a monthly report covering total income, total expense, net savings, savings rate, an expense breakdown by category, budget-vs-actual status per category, and

the top spending categories.

- **FR5 — Extensibility:** adding a new expense category or a new account should require minimal, additive code changes rather than modifying existing, tested logic.

Non-Functional Requirements

- **Data integrity:** the system must never silently accept a negative amount, an unrecognised category, an invalid date, or a reference to a non-existent account — each must be caught and reported as a distinct, specific error rather than crashing the whole batch of transactions or producing a wrong balance.
- **Usability:** reports must be human-readable, formatted plainly (fixed-width tables in a .txt file), so a non-technical user can open the report and understand it immediately.
- **Robustness:** one malformed row in an imported CSV file should not stop the rest of a batch import from succeeding — bad rows should be collected and reported separately.
- **Maintainability:** validation logic, data models, persistence, budgeting, and reporting should be separated into distinct modules so each can be tested and modified independently.

Key Design Constraints Identified

From the above requirements, three constraints drive the system design that follows in Section B: (1) transaction validation must happen at a single, well-defined boundary so every entry point — manual entry or CSV import — is equally protected; (2) budget limits must live in an external, editable JSON file rather than hardcoded values, so a user can change their budget without touching code; and (3) the reporting layer must depend only on the transaction and budget data, never on how that data was entered, so new report types can be added without touching the transaction-logging or validation code.

Conclusion

The core problem is not simply 'store some numbers' but building a small system with layered responsibilities: reliable multi-account transaction tracking, strict input validation with clear error reporting, configurable category-based budgeting, and analytics that turn raw transactions into decisions a user can act on. Sections B through E design and implement exactly these four responsibilities as separate, cooperating modules.